

Personalized Wordlists

To create a personalized wordlist for the user, we will need to collect some information about them. As our example here is a known public figure, we can check out their [Wikipedia page](#) or do a basic Google search to gather the necessary information. Even if this was not a known figure, we can still carry out the same attack and create a personalized wordlist for them. All we need to do is gather some information about them, which is discussed in detail in the [Hashcat](#) module, so feel free to check it out.

CUPP

Many tools can create a custom password wordlist based on certain information. The tool we will be using is **cupp**, which is pre-installed in your PwnBox. If we are doing the exercise from our own VM, we can install it with **sudo apt install cupp** or clone it from the [Github repository](#). **Cupp** is very easy to use. We run it in interactive mode by specifying the **-i** argument, and answer the questions, as follows:

```
Personalized Wordlists

dbcyph0n@htb[/htb]$ cupp -i

-----
cupp.py!                                # Common
\                                         # User
\                                         # Passwords
  (oo)____                                # Profiler
  (__)  )\
    ||--|| *    [ Muris Kurgas | j0rgan@remote-exploit.org ]
                  [ Mebus | https://github.com/Mebus/ ]

[+] Insert the information about the victim to make a dictionary
[+] If you don't know all the info, just hit enter when asked! ;)

> First Name: William
> Surname: Gates
> Nickname: Bill
> Birthdate (DDMMYYYY): 28101955

> Partners) name: Melinda
> Partners) nickname: Ann
> Partners) birthdate (DDMMYYYY): 15081964

> Child's name: Jennifer
> Child's nickname: Jenn
> Child's birthdate (DDMMYYYY): 26041996

> Pet's name: Nila
> Company name: Microsoft

> Do you want to add some key words about the victim? Y/[N]: Phoebe,Rory
> Do you want to add special chars at the end of words? Y/[N]: y
> Do you want to add some random numbers at the end of words? Y/[N]:y
> Leet mode? (i.e. leet = 1337) Y/[N]: y

[+] Now making a dictionary...
[+] Sorting list and removing duplicates...
[+] Saving dictionary to william.txt, counting 43368 words.
[+] Now load your pistolero with william.txt and shoot! Good luck!
```

And as a result, we get our personalized password wordlist saved as **william.txt**.


```
remote: Enumerating objects: 386, done.
remote: Total 386 (delta 0), reused 0 (delta 0), pack-reused 386
Receiving objects: 100% (386/386), 16.76 MiB | 5.38 MiB/s, done.
Resolving deltas: 100% (127/127), done.
```

This tool has many use cases that we can take advantage of to create advanced lists of potential usernames. However, for our simple use case, we can simply run it and provide the first/last names as arguments, and forward the output into a file, as follows:

Code: `bash`

`./username-anarchy Bill Gates > bill.txt`

We should finally have our username and passwords wordlists ready and we could attack the SSH server.

← Previous

Next →

+10 Streak pts

✔ Mark Complete & Next

 Cheat Sheet

Table of Contents

Introduction

Introduction to Brute Forcing

✔

Basic HTTP Auth Brute Forcing

- Password Attacks
- ✔
-  Default Passwords
- ✔
-  Username Brute Force
- ✔

Web Forms Brute Forcing

- Hydra Modules
- ✔
- Determine Login Parameters
- ✔
-  Login Form Attacks
- ✔

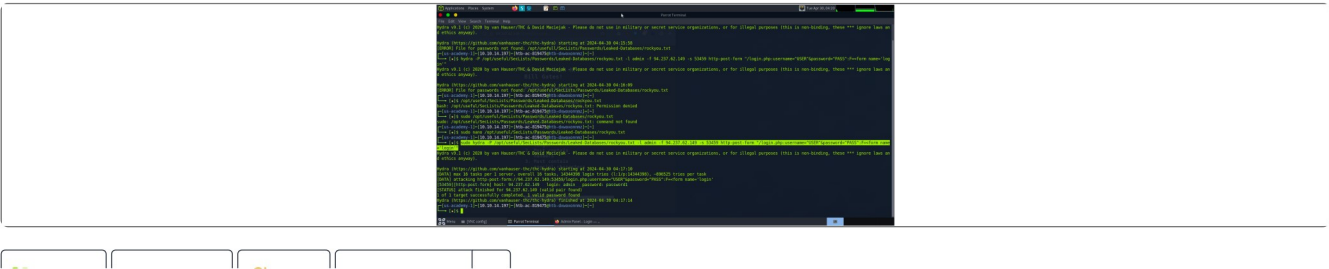
Service Authentication Attacks

- Personalized Wordlists
-  Service Authentication Brute Forcing

Skills Assessment

-  Skills Assessment - Website
-  Skills Assessment - Service Login

My Workstation



 Interact

 Terminate

 Reset

Life Left: 100m

